

Smali By bithowl: Chapter 8 Methods and Fields

("The Memory and Machinery Inside Every Android Class")



SMALI BY BITHOWL : CHAPTER 8

METHODS AND FIELDS

"The Memory and Machinery Inside Every Android Class"

```
.class public
Lcom/example/app/
AuthManager;
.super Ljava/lang/Object;
.field private token:
Ljava/lang/String;
.method public
login(Ljava/lang/String;)Z
...
.end method
```

THE STORY (HOOK)

You discover a locked room in an Android app...

**Safe**
Stores information

**Control Panel**
Performs actions

**Telephone**
Communicates outside

**Filing Cabinet**
Stores records

In programming:

FIELDS
→ data the class stores

- username
- token
- API_URL
- isLoggedIn

METHODS
→ actions the class performs

- login()
- logout()
- verifyToken()
- sendRequest()

These two pieces reveal the purpose of a class before reading its code.

WHAT IS A METHOD?

Java:

```
public boolean verifyToken(String token) {
    ...
}
```

Smali:

```
.method public verifyToken(Ljava/lang/String;)Z
...
.end method
```

Decoded Signature:

```
verifyToken → Parameter String → Return boolean
```

Signature = what goes in and what comes out.
Instructions inside = what actually happens.

STATIC vs INSTANCE METHODS

Type	Belongs to	Common Invocation
Static method	Class	invoke-static
Instance method	Object	invoke-virtual
Private / direct	Specific object / class context	invoke-direct

Static Method (example)

```
.method public static
isValid(Ljava/lang/String;)Z
...
.end method
```

Invoke:

```
invoke-static {v0},
Lcom/example/AuthUtils;->
isValid(Ljava/lang/String;)Z
```

Instance Method (example)

```
.method public verify
(Ljava/lang/String;)Z
...
.end method
```

Invoke:

```
invoke-virtual {p0, v0},
Lcom/example/AuthManager;->
verify(Ljava/lang/String;)Z
```

CONSTRUCTORS

Java:

```
public AuthManager(String token) { ... }
```

Smali:

```
.method public constructor
<init>(Ljava/lang/String;)V
...
.end method
```

- <init> is the constructor name.
- Return type is V (void).

Constructors reveal:

- ✓ Default configuration
- ✓ Security settings
- ✓ API endpoints
- ✓ Object dependencies
- ✓ Initial auth state
- ✓ Encryption config

CLASS INITIALIZER <clinit>

Smali:

```
.method static constructor <clinit>()V
...
.end method
```

Used to initialize static fields.

Example Java:

```
static {
    API_URL = "https://api.example.com";
}
```

Pay attention to what <clinit> initializes — it reveals configuration!

METHOD SIGNATURES

Examples:

Small Signature	Decoded	Java Equivalent
login()V	void	void login()
login(Ljava/lang/String;)Z	boolean	boolean login(String)
getToken(Ljava/lang/String;)String	String	String getToken()
calculate(II)I	int	int calculate(int,int)
login(Ljava/lang/String;Ljava/lang/String;)Z	boolean	boolean login(String,String)

Format:

```
methodName → (Param1) (Param2)... → ReturnType
```

PARAMETERS & RETURN VALUES

Parameters in Smali use parameter registers:

Register	Parameter
p0	this (for instance methods)
p1	1st parameter
p2	2nd parameter
...	...

Return Instructions:

- return-void → V (void)
- return v0 → Primitive (int, boolean, etc.)
- return-object v0 → Object (reference)

Return type is part of the signature.

WHAT IS A FIELD?

Java: private String token;

Smali: .field private token:Ljava/lang/String;

Access → **Name** → **Type**

private → token → String

Fields store data belonging to the class or object. Types can be primitives, objects, arrays, collections, etc.

STATIC vs INSTANCE FIELDS

Static Field (Class-level)

```
.field public static
API_URL:Ljava/lang/String;
```

One shared value for the entire class. Used for constants, config, global state, endpoints, singletons, feature flags.

Instance Field (Object-level)

```
.field private token:
Ljava/lang/String;
```

Each object has its own copy. Values differ across objects/instances.

ACCESSING FIELDS (INSTRUCTIONS)

Instance Fields

Read: iget / iget-object
Write: iput / iput-object

Example (iget-object):

```
iget-object v0, p0,
Lcom/example/User;->
token:Ljava/lang/String;
```

Reads the token field from the object in p0 and places it in v0.

iget/put → instance | sget/sput → static

Static Fields

Read: sget / sget-object
Write: sput / sput-object

Example (sget-object):

```
sget-object v0,
Lcom/example/Config;->
API_URL:Ljava/lang/String;
```

Reads the static API_URL field and places it in v0.

FIELD ACCESS MODIFIERS

Modifier	Access
public	Accessible from anywhere
private	Accessible only within the class
protected	Accessible to subclasses
static	Belongs to the class
final	Cannot be reassigned / overridden
volatile	Value may change across threads
transient	Not serialized
const (final static)	Compile-time constant

Modifiers appear in .field directives.

BUG HUNTER PERSPECTIVE

Trace a Field's Lifecycle

- Where is it initialized?
- Where is it written?
- Where is it read?
- Is it stored securely?
- Is it sent over the network?
- Can an attacker influence it?

Questions for Methods

- ✓ Who can call this method?
- ✓ Does it authenticate the caller?
- ✓ What parameters does it accept?
- ✓ Does it trust external input?
- ✓ What privileged action does it perform?

A suspicious field name is NOT a vulnerability.

Interesting code ≠ Vulnerability. Investigate context and behavior.

REALISTIC EXAMPLE

```
.class public Lcom/example/app/AuthManager;
.super Ljava/lang/Object;
.field private token:Ljava/lang/String;
```

- constructor <init>(Ljava/lang/String;)V
 - ↳ Receives String
- isAuthenticated()Z
 - ↳ Returns boolean
- getToken()Ljava/lang/String;
 - ↳ Returns String
- logout()V
 - ↳ Returns void

AuthManager
token : String

PRACTICAL EXERCISE

- Extract an APK: `apktool d app.apk`
- Choose a security-sensitive class (Auth, Login, Token, Crypto, Network, API, WebView, DB, etc.)
- Open the .smali file and create a table:

Item	What to identify
Fields	Name + type
Static fields	Class-level data
Instance fields	Object-level data
Methods	Name + signature
Constructors	<init>
Static initializer	<clinit>
Parameters	Input types
Return values	Output type

- Pick one field and one method.
- Trace where they are used.

Focus on one piece of data or one action. That is how you reverse engineer effectively.

QUICK RECAP

- ✓ Fields store data.
- ✓ Methods perform actions.
- ✓ Static methods belong to the class.
- ✓ Instance methods operate on objects.
- ✓ Constructors use <init>.
- ✓ Class initialization uses <clinit>.
- ✓ Method signatures show params & return.
- ✓ Static fields belong to class.
- ✓ Instance fields belong to objects.
- ✓ iget/put access instance fields.
- ✓ sget/sput access static fields.
- ✓ Access modifiers define visibility.
- ✓ Understand structure before chasing instructions.

FINAL THOUGHT

First ask: What does this class remember? → Look at the fields.
Then ask: What does this class do? → Look at the methods.
Connect the two. That's where vulnerabilities hide.

Security bugs live in the relationship between data (fields) and behavior (methods).

By - bithowl

A class without data is just an empty box. A class without behaviour is just a blueprint. Put them together, and you get something useful.

In Java, you already know them as:

- **Fields** → data the class stores
- **Methods** → actions the class performs

Smali has the same concepts.

But the way they're represented is much closer to the underlying bytecode. For an Android reverse engineer, understanding methods and fields is essential.

Because when you're hunting for a vulnerability, you're often asking two simple questions:

What does this class store?

and

What can this class do?

Fields answer the first question.

Methods answer the second.

Let's break them apart.

The Story (Hook)

Imagine you discover a locked room inside an Android application.

Inside the room you find:

A safe

A control panel

A telephone

A filing cabinet

The safe contains information.

The control panel performs actions.

The filing cabinet stores records.

The telephone communicates with other systems.

In programming terms:

Fields → Things the class stores

Methods → Things the class does

A field might contain:

username

token

API_URL

isLoggedIn

A method might:

login()

logout()

verifyToken()

sendRequest()

When reverse engineering an application, these two pieces often reveal the purpose of a class before you even understand its entire implementation.

What Is a Method?

A method represents a behaviour or operation.

Java:

```
public boolean verifyToken(String token) {  
    ...  
}
```

Smali:

```
.method public verifyToken(Ljava/lang/String;)Z  
    ...  
.end method
```

From Chapter 6, we can decode the signature:

verifyToken

|

|— Parameter: String

|

|— Return: boolean

The method signature tells us **what goes in and what comes out**.

The instructions inside the method tell us **what actually happens**.

Static Methods

A static method belongs to the class itself rather than a particular object.

Java:

```
public static boolean isValid(String token) {  
    ...  
}
```

Smali:

```
.method public static isValid(Ljava/lang/String;)Z  
  
    ...  
  
.end method
```

Notice: **public static**

The static keyword is part of the method's access flags. A static method can be called without creating an instance of the class.

For example: **AuthUtils.isValid(token);**

In Smali, you will commonly see static method calls using: **invoke-static**

Example: **invoke-static {v0}, Lcom/example/AuthUtils;->isValid(Ljava/lang/String;)Z**

This is an important distinction when reading bytecode.

Instance Methods

Instance methods belong to a particular object.

Java:

```
public boolean verify(String token) {  
    ...  
}
```

Smali:

```
.method public verify(Ljava/lang/String;)Z  
  
    ...  
  
.end method
```

Notice there is no: **static**

An instance method operates on an object instance.

For example:

```
AuthManager manager = new AuthManager();
```

```
manager.verify(token);
```

In Smali, instance method calls commonly use:

invoke-virtual

or:

invoke-direct

depending on the method and access context.

You'll learn these invocation instructions in much greater detail later.

Static vs Instance Methods

The difference is simple:

Type	Belongs to	Common invocation
Static method	Class	invoke-static
Instance method	Object	invoke-virtual
Private/direct method	Specific object/class context	invoke-direct

Example: **MathUtils.calculate();**

This doesn't require an object.

But: **user.getToken();**

operates on a particular user object.

Understanding this distinction becomes extremely important when tracing execution through Smali.

Constructors

Constructors are special methods.

In Java:

```
public AuthManager() {  
    ...  
}
```

In Smali:

```
.method public constructor <init>()V  
    ...  
.end method
```

The unusual part is: <init>

That's the constructor name in Smali.

The return type is: V meaning void.

Constructors initialize objects.

For example:

```
public AuthManager(String token) {  
    this.token = token;  
}
```

could appear conceptually as:

```
.method public constructor <init>(Ljava/lang/String;)V  
  
...  
  
.end method
```

Constructors are particularly interesting during reverse engineering because initialization logic can reveal:

- Default configuration
- Security settings
- API endpoints
- Object dependencies
- Initial authentication state
- Encryption configuration

Static Constructors

There's another special method: <clinit>

Example:

```
.method static constructor <clinit>()V  
  
...  
  
.end method
```

This is the class initializer. It is commonly used to initialize static fields.

For example:

```
static {  
    API_URL = "https://api.example.com";  
}
```

may be represented through <clinit>.

When you see <clinit>, pay attention to what it initializes.

It can provide useful clues about how the application configures itself.

Method Signatures

A method signature tells you:

- Method name
- Parameter types
- Return type

Example:

```
.method public login(Ljava/lang/String;Ljava/lang/String;)Z
```

Decode it:

Name: login

Parameter 1: String

Parameter 2: String

Return: boolean

Equivalent Java: public boolean login(String username, String password)

Another: .method public getToken()Ljava/lang/String;

means: public String getToken()

And: .method private calculate(II)I

means: private int calculate(int a, int b)

Once you become comfortable with descriptors, method signatures become surprisingly easy to read.

Parameters

Methods can accept parameters.

Example: .method public login(Ljava/lang/String;)Z

The method accepts one String.

Multiple parameters appear sequentially: .method public login(Ljava/lang/String;Ljava/lang/String;)Z

That's: String

String

↓

boolean

In the method body, parameters are commonly represented using **parameter registers** such as:

p0

p1

p2

For instance methods, p0 commonly represents this reference.

So when you see: p0

don't automatically assume it is the first declared method argument. For an instance method, it generally represents the current object.

This distinction becomes critical when analysing registers.

Return Values

Methods can return different types.

Void

```
.method public logout()V
```

V means the method returns nothing.

The method typically ends with:

```
return-void
```

Integer / Boolean

```
.method public isValid()Z
```

The result is returned using an instruction such as:

```
return v0
```

Object

```
.method public getToken()Ljava/lang/String;
```

An object return commonly uses: return-object v0

Understanding return types helps you follow data through the application.

What Is a Field?

A field is data associated with a class or an object.

Java: private String token;

Smali: .field private token:Ljava/lang/String;

The field contains:

Access: private

Name: token

Type: String

Fields can store almost anything: Strings, Integers, Booleans, Objects, Arrays, Collections, References

Static Fields

A static field belongs to the class rather than individual objects.

Java: `public static String API_URL;`

Smali: `.field public static API_URL:Ljava/lang/String;`

There is only one class-level value rather than one value per object instance.

Static fields are often used for:

- Constants
- Configuration
- Global state
- API endpoints
- Singleton instances
- Feature flags

For a bug hunter, static fields can be interesting because developers sometimes place sensitive configuration there.

For example: `.field private static API_KEY:Ljava/lang/String;`

This is worth investigating.

But remember: **A suspicious field name is not automatically a vulnerability.**

You need to determine what the value contains and how it is used.

Instance Fields

Instance fields belong to individual objects.

Example:

`.field private username:Ljava/lang/String;`

`.field private token:Ljava/lang/String;`

`.field private loggedIn:Z`

Equivalent Java:

`private String username;`

`private String token;`

`private boolean loggedIn;`

Imagine two objects:

User A

└─ username = "alice"

└─ token = "..."

└─ loggedIn = true

User B

└─ username = "bob"

└─ token = "..."

└─ loggedIn = false

Each object has its own copy of the instance fields.

Static vs Instance Fields

This distinction is fundamental.

Static Field

|



Belongs to the class

|

└─ One shared class-level value

Instance Field

|



Belongs to an object

|

└─ Each object has its own value

Example:

.field public static APP_VERSION:Ljava/lang/String;

.field private username:Ljava/lang/String;

The first is class-level.

The second belongs to an instance.

Accessing Fields in Smali

Smali uses specific instructions to read and write fields.

For instance fields, you'll commonly encounter:

iget

iput

For static fields:

sget

sput

For example:

```
iget-object v0, p0, Lcom/example/User;->token:Ljava/lang/String;
```

This means:

Read the token field from the object represented by p0 and place the result into v0.

And:

```
sget-object v0, Lcom/example/Config;->API_URL:Ljava/lang/String;
```

means:

Read the static API_URL field and place it into v0.

We'll study these instructions in detail when we get into Smali operations.

For now, remember the pattern:

iget / iput → instance fields

sget / sput → static fields

Field Access Modifiers

Fields can have access modifiers.

Example: `.field public username:Ljava/lang/String;`

Public means the field is accessible according to normal Java visibility rules.

Private: `.field private token:Ljava/lang/String;`

Protected: `.field protected sessionId:Ljava/lang/String;`

Static: `.field private static API_URL:Ljava/lang/String;`

Final: `.field private static final VERSION:I = 0x1`

These modifiers tell you how the field is intended to be used.

Bug Hunter Perspective

This is where methods and fields become especially interesting for security research.

Imagine you find:

```
.field private token:Ljava/lang/String;
```

Don't immediately conclude:

"Token vulnerability!"

Instead, trace the field.

Ask:

Where is token initialized?



Where is token written?



Where is token read?



Is it stored securely?



Is it sent over the network?



Can an attacker influence it?

The same approach applies to methods.

Suppose you find: `.method public resetPassword(Ljava/lang/String;)V`

The method name sounds sensitive.

But you need to investigate:

- Who can call it?
- Does it authenticate the caller?
- What parameters does it accept?
- Does it trust external input?
- What privileged action does it perform?

This is the difference between **finding interesting code** and **finding a vulnerability**.

A Realistic Example

Consider:

```
.class public Lcom/example/app/AuthManager;
```

```
.super Ljava/lang/Object;
```

```
.field private token:Ljava/lang/String;
```

```
.method public constructor <init>(Ljava/lang/String;)V
```

```

...
.end method

.method public isAuthenticated()Z
...
.end method

.method public getToken()Ljava/lang/String;
...
.end method

.method public logout()V
...
.end method

```

We can immediately build a mental model:

AuthManager

```

|
|— Field
|   └─ token : String
|
|— Constructor
|   └─ Receives String
|
|— isAuthenticated()
|   └─ Returns boolean
|
|— getToken()
|   └─ Returns String
|
└─ logout()
    └─ Returns void

```

We haven't read the method bodies yet. But we've already learned a lot about the class.

That's the power of understanding structure.

Practical Exercise Method & Field Mapping

Take an APK and choose a security-sensitive class.

Good candidates include classes containing words such as:

Auth

Login

Token

Crypto

Network

Api

WebView

Database

Security

Open the corresponding Smali file.

Create a simple table:

Item	What to identify
Fields	Name + type
Static fields	Class-level data
Instance fields	Object-level data
Methods	Name + signature
Constructors	<init>
Static initializer	<clinit>
Parameters	Input types
Return values	Output type

Then choose one interesting field and one interesting method.

Trace where they are used.

Don't try to understand the entire application. Learn to follow **one piece of data or one action** through the code. That skill will become extremely valuable later.

Quick Recap

- **Fields store data.**
- **Methods perform actions.**
- Static methods belong to the class.
- Instance methods operate on objects.
- Constructors use <init>.
- Static class initialization uses <clinit>.
- Method signatures describe parameters and return values.
- p0, p1, etc. commonly represent method parameters.
- Static fields belong to the class.
- Instance fields belong to individual objects.
- iget / iput commonly access instance fields.
- sget / sput commonly access static fields.
- Access modifiers reveal intended visibility and usage.

Final Thought

When you open a Smali class, don't immediately chase instructions.

First ask two questions: **What does this class remember?**

Look at the fields.

What does this class do?

Look at the methods.

Then connect the two.

A field stores the data.

A method changes, reads, validates, or sends that data.

And somewhere between those two...

is often where the vulnerability lives.

Because security bugs aren't just hidden in instructions.

They're hidden in the **relationship between data and behaviour.**

By - bithowl